



WS & SOA (IT644)

CodeTrail

Name	Student ID
Monar	202512019
Rishika Shah	202512041
Meet Sheth	202512057
Ruchita Patadiya	202512070
Shaikh Maryam	202512074
Mahi Kansara	202512078
Anistina Dsouza	202512081

CodeTrail

Build Together. Prove Every Contribution.

Requirement Specification Document

1. Introduction

1.1 Purpose

This Requirements Specification Document defines the requirements for CodeTrail, a web-based real-time collaborative code editor with integrated contribution tracking and Proof-of-Work logging.

CodeTrail enables multiple users to write, edit, execute, and manage source code together in real time through a shared online workspace. In addition to collaborative coding, the system automatically records significant coding activities of individual users and maintains a contribution history.

The system is designed to improve transparency, accountability, and collaboration in academic projects, hackathons, freelance development, remote software teams, and technical interviews.

2. Project Scope

The scope of CodeTrail is to develop a browser-based collaborative coding platform that allows authenticated users to work together on source code while automatically tracking their individual contributions.

The system will provide the following major capabilities:

- User registration and login.
- Secure authentication and authorization.
- User profile management.
- Creation of coding sessions.
- Joining collaborative coding sessions.
- Session and workspace management.
- Real-time collaborative code editing.
- Live cursor synchronization.
- File management.
- Multi-language code execution.
- Execution output and error display.
- In-session communication.
- Real-time notifications.
- Automatic activity logging.
- Proof-of-Work logging.
- SHA-256 hash-based activity verification.
- Contribution tracking.
- Contribution analytics.
- Session history.
- Contribution report generation.
- PDF and JSON report export.
- Report verification.
- User dashboard.
- Administrator dashboard.

- User management.
- Session monitoring.
- Platform analytics.

The system is focused on browser-based collaborative programming and contribution tracking. It is not intended to replace complete desktop IDEs, advanced version-control systems, or enterprise project-management systems.

3. Objectives

The main objectives of CodeTrail are:

1. To provide a real-time collaborative coding environment.
2. To allow multiple users to work on the same codebase simultaneously.
3. To provide browser-based code execution.
4. To automatically record significant coding activities.
5. To maintain a tamper-evident Proof-of-Work history.
6. To provide transparent individual contribution tracking.
7. To generate contribution analytics.
8. To generate verifiable contribution reports.
9. To provide secure authentication and authorization.
10. To provide administrators with tools for managing users and monitoring sessions.

4. Target Users

4.1 Students and Academic Educators

Students can use CodeTrail for collaborative programming assignments, academic projects, and hackathons.

Educators can use the platform to evaluate individual contributions and generate verifiable contribution reports.

Primary Needs

- Browser-based collaborative coding.
- Fair contribution tracking.
- Easy project setup.
- Verifiable activity reports.

4.2 Freelance Developers and Clients

Freelancers can use CodeTrail to collaborate with clients and maintain transparent development records.

Clients can monitor development progress and verify completed work.

Primary Needs

- Transparent development tracking.
- Proof of completed work.
- Secure collaboration.
- Milestone verification.

4.3 Remote Development Teams and Technical Interviewers

Remote development teams can use CodeTrail for collaborative development and pair programming.

Technical interviewers can use the platform for live coding and session review.

Primary Needs

- Real-time collaboration.
- Multi-language execution.

- Session review.
- Team contribution insights.

4.4 Platform Administrators

Administrators are responsible for monitoring and managing the platform.

Primary Needs

- User management.
- Session monitoring.
- Resource management.
- System analytics.

5. System Overview

CodeTrail is a web application consisting of:

- Frontend interface.
- Backend server.
- Database.

- Authentication system.
- Real-time communication layer.
- Collaborative code editor.
- Code execution service.
- Activity logging system.
- Contribution analytics module.
- Report generation module. ● Administration module.

The frontend provides the user interface and collaborative workspace. The backend handles authentication, APIs, session management, activity logging, workspace operations, and reporting.

MongoDB stores application data, while Socket.IO provides real-time communication between connected users.

6. Functional Requirements

FR-01: User Registration

The system shall allow new users to create an account.

The system shall:

- Provide a registration interface.
- Accept required user information.

- Validate input.
- Check required fields.
- Create a user account after successful validation.
- Securely store user credentials.

FR-02: User Login

The system shall allow registered users to log into CodeTrail.

The system shall:

- Accept login credentials.
- Validate credentials.
- Authenticate the user.
- Generate a JWT token after successful authentication.
- Redirect the authenticated user to the appropriate application area.
- Reject invalid credentials.

JWT is specified as the authentication mechanism in the project technology stack.

FR-03: User Profile

The system shall allow authenticated users to:

- View their profile.
- Update permitted profile information.

-
-

View their sessions.

View their contribution information.

FR-04: Role-Based Authorization

The system shall provide role-based access.

The system shall distinguish between:

- Session Owner.
- Collaborator.
- Administrator.

Users shall only be allowed to perform operations permitted by their role.

FR-05: Create Coding Session

The system shall allow a Session Owner to create a collaborative coding session.

The system shall:

- Generate a unique session.
- Create a workspace.
- Assign the creator as the Session Owner.
- Store session information.
- Allow collaborators to be invited.

FR-06: Join Coding Session

The system shall allow authorized users to join an existing coding session.

The system shall:

-
-

- Authenticate the user.
- Verify access permissions.
- Load the shared workspace.
- Add the user to the participant list.
 - Notify existing participants.
 - Record the user's participation.

FR-07: Real-Time Collaborative Editing

The system shall allow multiple users to edit the same codebase simultaneously.

The system shall:

- Synchronize code changes.
- Display changes made by other users.
- Maintain a shared workspace.
- Support simultaneous participants. ● Maintain collaboration status.

Socket.IO is specified for real-time collaboration and synchronization.

FR-08: Live Cursor Synchronization

The system shall synchronize cursor positions between users.

The system shall:

- Track active users.
- Display collaborator information.
- Synchronize cursor positions.
- Update participant status in real time.

-
-

FR-09: Collaborative Code Editor

The system shall provide an online code editor.

The editor shall support:

- Syntax highlighting.
- Auto-completion.
Bracket matching.
Multiple programming languages.
- Multiple themes.
- Real-time code editing.

Monaco Editor is specified as the code editor technology.

FR-10: File Management

The system shall allow authorized users to manage files within a workspace.

Users shall be able to:

- Create files.
- Open files.
- Edit files.
- View files.
- Organize workspace files.

FR-11: Code Execution

The system shall allow users to execute code directly from the browser.

The execution process shall:

-
-

1. Receive source code.
2. Receive the selected language.
3. Send the execution request to the backend.
4. Forward the request to the execution service.
5. Receive the result.
6. Display the result to the user.
7. Record the execution activity.

The project specifies Piston API for code execution and identifies languages including C, C++, Java, Python, JavaScript, Go, Rust, and Kotlin.

FR-12: Execution Output The

system shall display:

- Program output.
- Compilation errors.
- Runtime errors.
- Execution errors.

FR-13: Activity Logging

The system shall automatically record significant coding activities.

Activities shall include:

- Code edits.
- File changes.
- Code execution.
- Session joins. ● Session exits.
- Participation. ● Timestamps.

The project specifically defines automatic logging of significant user activities.

FR-14: Proof-of-Work Logging

The system shall maintain a cryptographically linked activity history.

The system shall:

- Group activities into blocks.
- Generate SHA-256 hashes.
- Link activity blocks.
- Store activity records securely.
- Support verification of the activity history.

The project specifies SHA-256 hash chaining so that modification of an earlier record can be detected.

FR-15: Contribution Tracking

The system shall automatically track individual contributions.

The system shall maintain:

- User-wise coding activity.
- Execution activity.
- Participation information.
- Activity timestamps. ● Contribution history.

FR-16: Contribution Analytics

The system shall generate contribution statistics.

The analytics module shall provide:

- User-wise contribution summaries.
- Coding activity.
- Participation statistics.
- Execution statistics.
- Session-based contribution information.

FR-17: Session History

The system shall maintain the history of coding sessions.

Users shall be able to:

- View previous sessions.
- Search sessions.
- Filter sessions.
- Access previous reports.

The project specifies dashboard functionality for recent sessions, collaboration history, session search/filtering, and previous reports.

FR-18: In-Session Chat

The system shall provide chat functionality within a collaborative session.

Users shall be able to:

- Send messages.
- Receive messages.
- Communicate with collaborators.

FR-19: Real-Time Notifications

The system shall provide notifications for important collaboration events.

Notifications shall include:

- User joins.
- User leaves.
- Code execution.
- Collaboration status changes.

FR-20: Report Generation

The system shall generate structured contribution reports.

Reports shall include, where applicable:

- Participant information.
- Session information.
- Activity summaries.
- Timestamps.
- Participation details.
- Coding activity.
- Execution history.
- Workspace information.

FR-21: Report Export

The system shall allow authorized users to export contribution reports.

The system shall support:

- PDF export.

- JSON export. jsPDF is specified for PDF report generation.

FR-22: Report Verification

The system shall allow authorized users to verify contribution records.

The system shall:

- Retrieve relevant activity records.
- Check cryptographic hashes.
- Validate the hash chain.
- Identify whether records have been modified.
- Display the verification result.

FR-23: User Dashboard

The system shall provide a dashboard containing:

- Recent sessions.
- Active sessions.
- Previous sessions.
- Contribution information.
- Reports.
- Collaboration history.

FR-24: Session Management Session

Owners shall be able to:

- Create sessions.
- Edit sessions.
- Delete sessions.
- Invite collaborators.
- Manage active sessions.
- Manage completed sessions.
- Access session history.

FR-25: Administrator Management

Administrators shall be able to:

- View users.
- Manage users.
- Monitor sessions.
- View system statistics.
- Monitor platform activity.
- Manage platform resources.

7. Non-Functional Requirements

NFR-01: Performance

The system should provide responsive interaction during collaborative coding.

Real-time code changes, cursor movements, participant updates, and notifications should be transmitted with minimal delay.

NFR-02: Scalability

The system should support multiple simultaneous users and collaborative coding sessions.

NFR-03: Security The

system shall provide:

- JWT authentication.
- Password hashing.
- Protected session access.
- Input validation.
- HTTP security headers.
- CORS protection.
- Secure database storage.
- Cryptographic activity logging.

The project specifies JWT, bcrypt, Helmet, CORS, and input validation as security mechanisms.

NFR-04: Usability The

interface shall be:

- Simple.
- Intuitive.
- Consistent.
- Easy to navigate.
- Easy to understand.

Users should be able to access coding, execution, collaboration, and contribution features without unnecessary complexity.

NFR-05: Reliability

The system should reliably preserve:

- User information.

- Session information.
- Workspace files.
- Source code.
- Activity logs.
- Contribution records. ● Reports.

NFR-06: Data Integrity

The system shall maintain the integrity of contribution records through cryptographic hashing.

NFR-07: Compatibility

The system shall operate through modern web browsers without requiring separate desktop software.

NFR-08: Maintainability

The application should use modular components for:

- Authentication.
- User management.
- Workspace management.
- Collaboration.
- Code execution.
- Activity logging.
- Analytics.
- Reporting.
- Administration.

NFR-09: Responsiveness

The web interface should adapt to different screen sizes.

However, the main collaborative coding workspace is primarily designed for desktop/laptop use because coding requires a keyboard and a multi-pane interface.

NFR-10: Extensibility

The architecture should allow future programming languages and additional collaboration, analytics, and reporting features.

8. Detailed User Stories

8.1 Authentication

US-01 – Register

As a new user, I want to create an account so that I can use CodeTrail.

Acceptance Criteria:

- Registration form is available.
- Required information is validated.
- Invalid information is rejected.
- A valid registration creates an account.

US-02 – Login

As a registered user, I want to log in so that I can access my sessions and workspace.

Acceptance Criteria:

- Valid credentials allow login.
- Invalid credentials are rejected.
- An authenticated session is created.
- Protected pages cannot be accessed without authentication.

8.2 Session Management

US-03 – Create Session

As a Session Owner, I want to create a coding session so that I can collaborate with other users.

Acceptance Criteria:

- A session can be created.
- The creator becomes the owner.
- A workspace is created.
- Collaborators can be invited.

US-04 – Join Session

As a collaborator, I want to join an authorized session so that I can work with my team.

Acceptance Criteria:

- User authentication is checked.
- Session access is verified.
- Workspace loads successfully.
- User appears among the participants.

US-05 – Manage Session

As a Session Owner, I want to manage my coding session so that I can control my workspace.

Acceptance Criteria:

- Owner can edit session information.
- Owner can invite collaborators.
- Owner can delete the session.
- Unauthorized users cannot perform owner-level actions.

8.3 Collaborative Coding

US-06 – Edit Code

As a developer, I want to edit shared code in real time so that multiple team members can work together.

Acceptance Criteria:

- User can edit code.
- Changes are synchronized.
- Other participants see the changes without refreshing.

US-07 – View Collaborators

As a developer, I want to see other active users so that I know who is currently working in the session.

Acceptance Criteria:

- Active users are displayed.
- Participant status is updated.
- Cursor positions are synchronized.

US-08 – Manage Files

As a developer, I want to create and modify workspace files so that I can organize my project code.

Acceptance Criteria:

- User can create files.
- User can edit files.
- Files are available to authorized collaborators.

8.4 Code Execution

US-09 – Run Code

As a developer, I want to execute my code in the browser so that I can test it immediately.

Acceptance Criteria:

- Language can be selected.
- Code can be executed.
- Output is displayed.
- Execution activity is recorded.

US-10 – View Errors

As a developer, I want to see execution errors so that I can identify and fix problems in my program.

Acceptance Criteria:

- Compilation errors are displayed.
- Runtime errors are displayed.
- Execution failures are communicated clearly.

8.5 Contribution Tracking

US-11 – Automatic Activity Logging

As a user, I want my coding activities to be automatically recorded so that my contribution can be verified.

Acceptance Criteria:

- Code edits are logged.
- Code executions are logged.
- Session participation is logged.
- Timestamps are stored.

US-12 – View Contribution

As a user, I want to view my contribution history so that I can understand my participation.

Acceptance Criteria:

- Contribution history is available.
- Activities are associated with the correct user.
- Contribution statistics are displayed.

US-13 – Verify Contribution

As an educator, client, or team leader, I want to verify contribution records so that I can confirm their authenticity.

Acceptance Criteria:

- Records can be retrieved.
- Hash integrity can be checked. ● Tampering can be detected.

8.6 Reporting

US-14 – Generate Report

As an educator, client, or team leader, I want to generate a contribution report so that I can evaluate project participation.

Acceptance Criteria:

- Report can be generated.
- Participant information is included.
- Activity information is included.
- Timestamps are included.

US-15 – Export Report

As an authorized user, I want to export a report so that I can store or share contribution information.

Acceptance Criteria:

- PDF report can be generated.
- JSON report can be generated.
- Exported report contains contribution information.

8.7 Administration

US-16 – Manage Users

As an administrator, I want to manage users so that I can maintain the platform.

Acceptance Criteria:

- Administrator can view users.
- Administrator can manage users.
- Normal users cannot access administrative functions.

US-17 – Monitor Sessions

As an administrator, I want to monitor active sessions so that I can oversee platform usage.

Acceptance Criteria:

- Active sessions are visible.
- Participants can be monitored.
- Platform statistics are available.

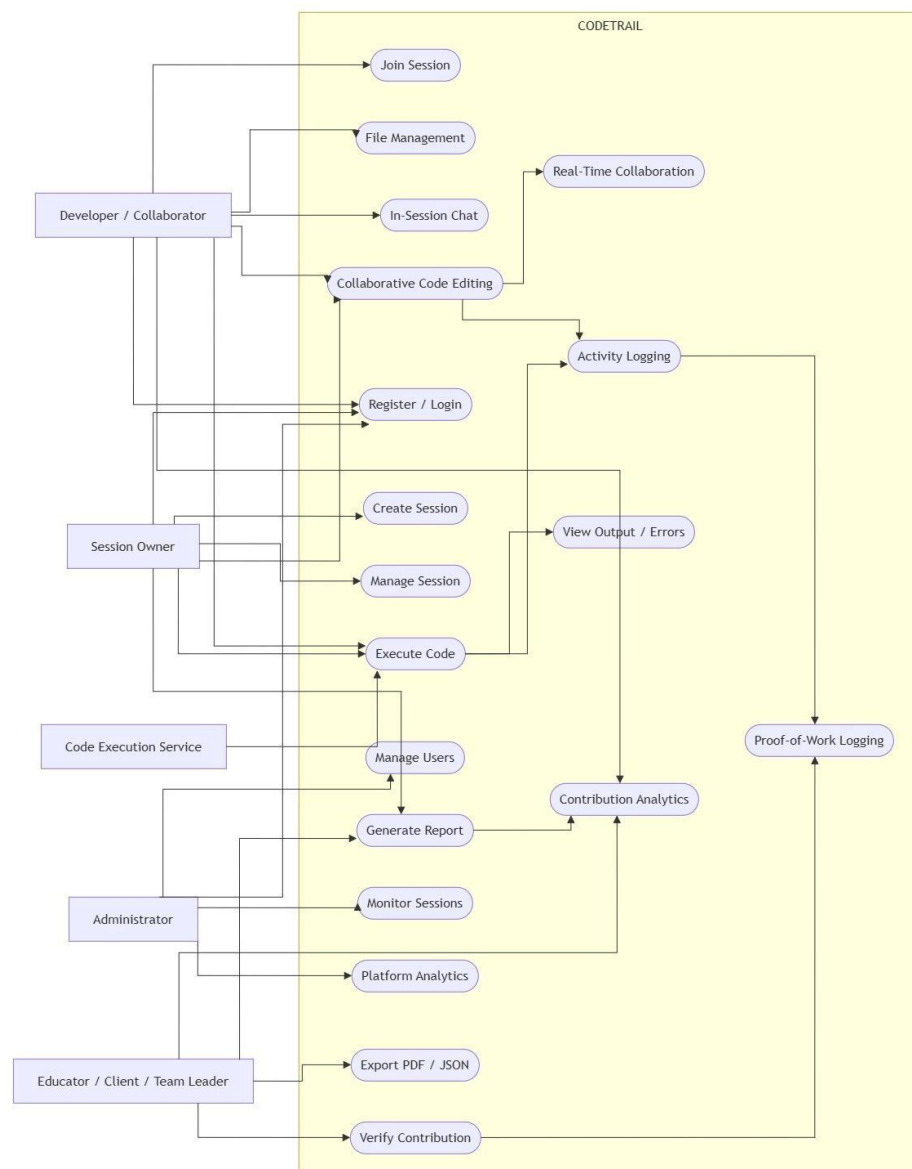
9. Use Case Diagram

9.1 Actors

The primary actors are:

- Developer / Collaborator
- Session Owner
- Team Leader / Educator / Client
- Administrator
- Code Execution Service
- System

9.2 Use Case Diagram



The five primary use cases specified in the project are:

1. Real-Time Collaborative Workspace.
2. Multi-Language Code Execution.
3. Automated Proof-of-Work Logging.
4. Audit Report and Verification.
5. Platform Administration.

10. Detailed Use Case Specifications

UC-01: Real-Time Collaborative Workspace

Primary Actor: Developer / Collaborator

Goal:

Allow multiple users to work on the same codebase simultaneously.

Preconditions

- User is registered.
- User is authenticated.
- User is authorized to access the session.

Main Flow

1. User joins a collaboration room.
2. System authenticates the user.
3. System verifies access.
4. Shared workspace loads.
5. Code editor is displayed.
6. User edits code.
7. Changes are synchronized.
8. Other participants see the changes.
9. Cursor and participant information are updated.
10. Activity is recorded.

Alternative Flow

- Invalid authentication → access denied.
- Unauthorized session → access denied.
- Real-time connection failure → system displays an appropriate error.

Postcondition

The user's changes are reflected in the shared workspace and contribution history.

UC-02: Multi-Language Code Execution

Primary Actor: Developer

Preconditions

- User is authenticated.
- User is inside an authorized session.
- Code is available.
- A supported language is selected.

Main Flow

1. User selects a programming language.
2. User enters or edits code.
3. User clicks Run.
4. Backend receives the request.
5. Backend sends the request to the execution service.
6. Code is compiled/executed.
7. Result is returned.
8. Output/error is displayed.
9. Execution is logged.

Alternative Flow

- Unsupported language → error displayed.
- Compilation failure → compilation error displayed.

- Runtime failure → runtime error displayed.
- Execution service unavailable → execution failure displayed.

Postcondition

The execution result is displayed and the execution activity is recorded.

UC-03: Automated Proof-of-Work Logging

Primary Actor: System

Main Flow

1. User performs a significant activity.
2. System captures the activity.
3. User and session are identified.
4. Timestamp is generated.
5. Activity is grouped into a block.
6. SHA-256 hash is generated.
7. Block is linked with the previous block.
8. Record is stored securely.

Postcondition

The activity becomes part of the tamper-evident Proof-of-Work history.

UC-04: Audit Report and Verification

Primary Actor: Team Leader / Educator / Client

Main Flow

1. User opens analytics.
2. System retrieves contribution records.
3. Contribution statistics are generated.
4. User generates a report.
5. Report is exported.
6. User requests verification.
7. System validates the cryptographic records.

8. Verification result is displayed.

Postcondition

The authorized user receives a contribution report and can verify its authenticity.

UC-05: Platform Administration

Primary Actor: Administrator

Main Flow

1. Administrator logs in.
2. System authenticates administrator.
3. Dashboard displays platform statistics.
4. Administrator views users.
5. Administrator manages users.
6. Administrator monitors sessions.
7. Administrator reviews platform activity.
8. Administrative actions are recorded.

Postcondition

The platform is monitored and managed by an authorized administrator.

11. Interface Requirements

11.1 General User Interface The

CodeTrail interface shall provide:

- Application navigation.
- Authentication screens.
- Dashboard.
- Session creation/joining.
- Collaborative workspace.
- File explorer.
- Code editor.
- Run/execute controls.
- Output console.
- Collaborator information.
- Chat.
- Contribution analytics.
- Report generation.
- Administration interface.

11.2 Authentication Interface The

login interface shall contain:

- CodeTrail branding.
- Email/username input.
- Password input.
- Login button.
- Registration navigation.
- Error/validation messages.

The registration interface shall contain:

- Name input.

- Email input.
- Password input.
- Confirm password input.
- Registration button.
- Login navigation.
- Validation messages.

11.3 Dashboard Interface The dashboard

shall provide access to:

- User information.
- Active sessions.
- Recent sessions.
- Create Session.
- Join Session.
- Contribution information.
- Reports.
- Session history.

11.4 Coding Workspace Interface The main

collaborative workspace shall contain:

- Session information.
- Navigation/actions.
- File explorer.
- Code editor.
- Run button.
- Language selector.
- Collaborator list.
- Live cursor information.
- Output console.
- Chat interface.

11.5 Analytics Interface

The analytics interface shall contain:

- Session information.
- Participant information.
- Contribution statistics.
- Coding activity.
- Execution activity.
- Activity history.
- Report generation.
- Report export.
- Verification functionality.

11.6 Administration Interface The

administrator interface shall contain:

- Platform statistics.
- User management.
- Session monitoring.
- Activity information.
- System analytics.

12. External Interface Requirements

12.1 Frontend-Backend Interface

The frontend shall communicate with the backend through REST APIs.

The APIs shall support:

- Authentication.
- User management.
- Session management.
- Workspace management.
- File management.
- Activity logging.
- Contribution retrieval. ● Report generation.

Express.js will be used for backend REST APIs.

12.2 Real-Time Interface

Socket.IO shall provide real-time communication for:

- Code synchronization.
- Cursor synchronization.
- Participant updates.
- Notifications. ● Chat.

12.3 Code Execution Interface

The backend shall communicate with the Piston API for code compilation and execution.

12.4 Database Interface

The application shall communicate with MongoDB through Mongoose.

MongoDB will store user information, workspaces, files, activity logs, reports, and notifications.

13. Initial UI/UX Wireframes

13.1 Wireframe Design Approach

The initial wireframes are based on the functional structure of CodeTrail rather than being final visual designs.

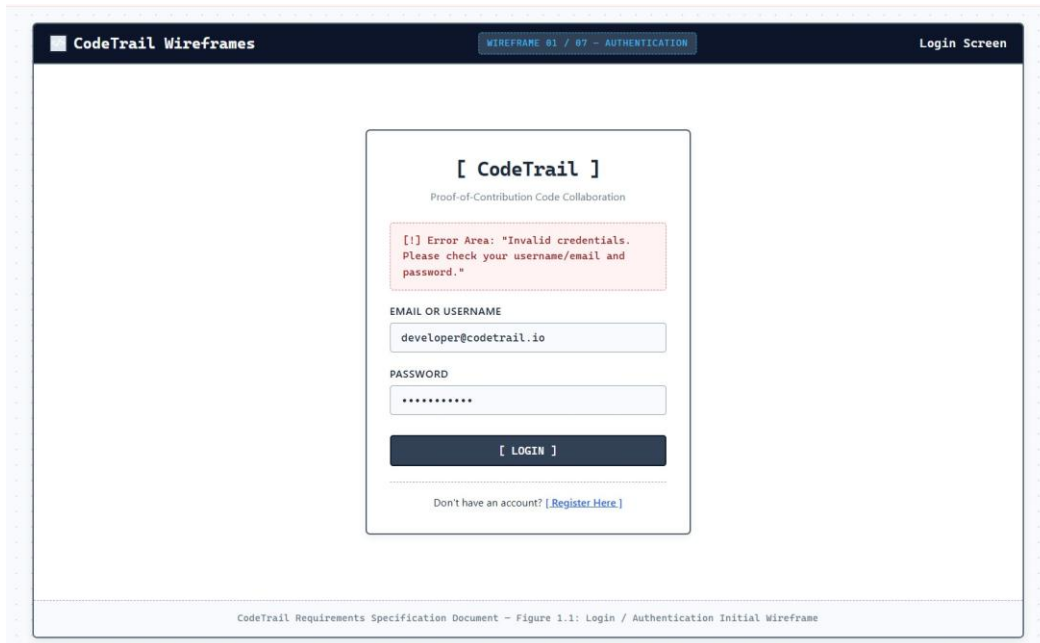
The wireframes represent the major screens required by the deployed web application:

1. Landing/Login interface.
2. Registration interface.
3. User dashboard.
4. Collaborative coding workspace.
5. Contribution analytics.
6. Report interface.

7. Administrator dashboard.

The final visual design may differ from these low-fidelity wireframes.

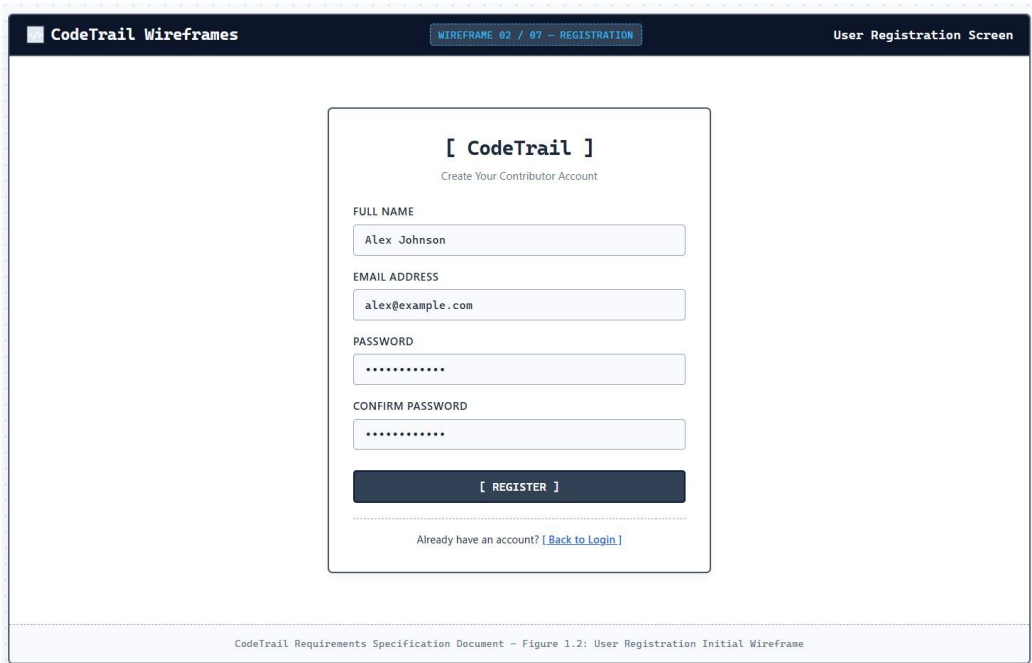
13.2 Wireframe – Login Page



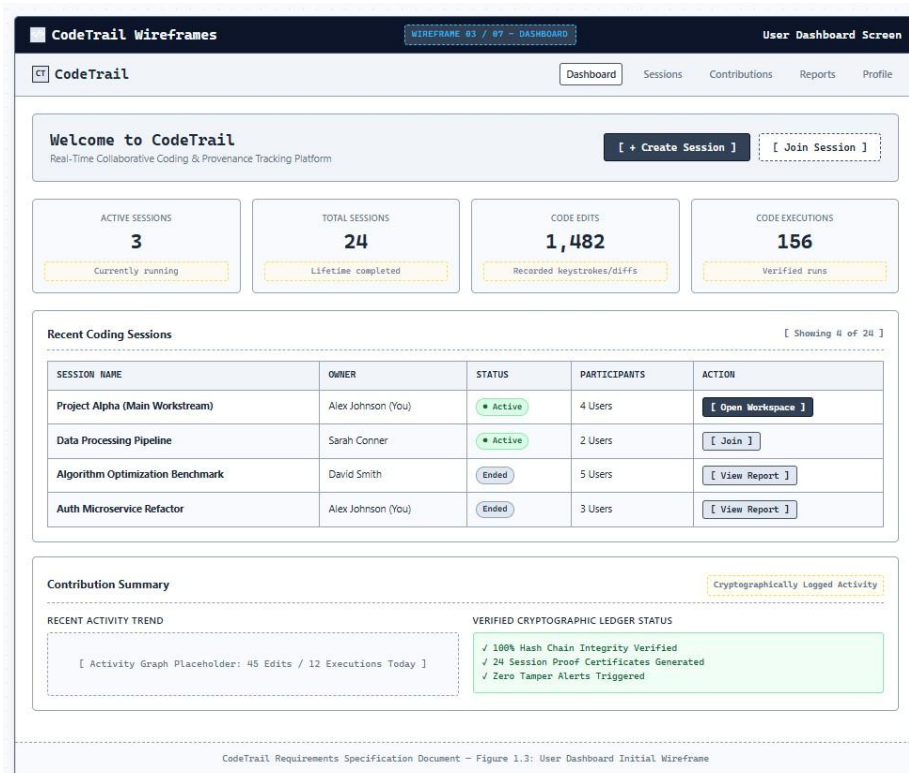
Main Elements

- CodeTrail branding.
- Login form.
- Authentication controls.
- Registration navigation. ● Error message area.

13.3 Wireframe – Registration Page

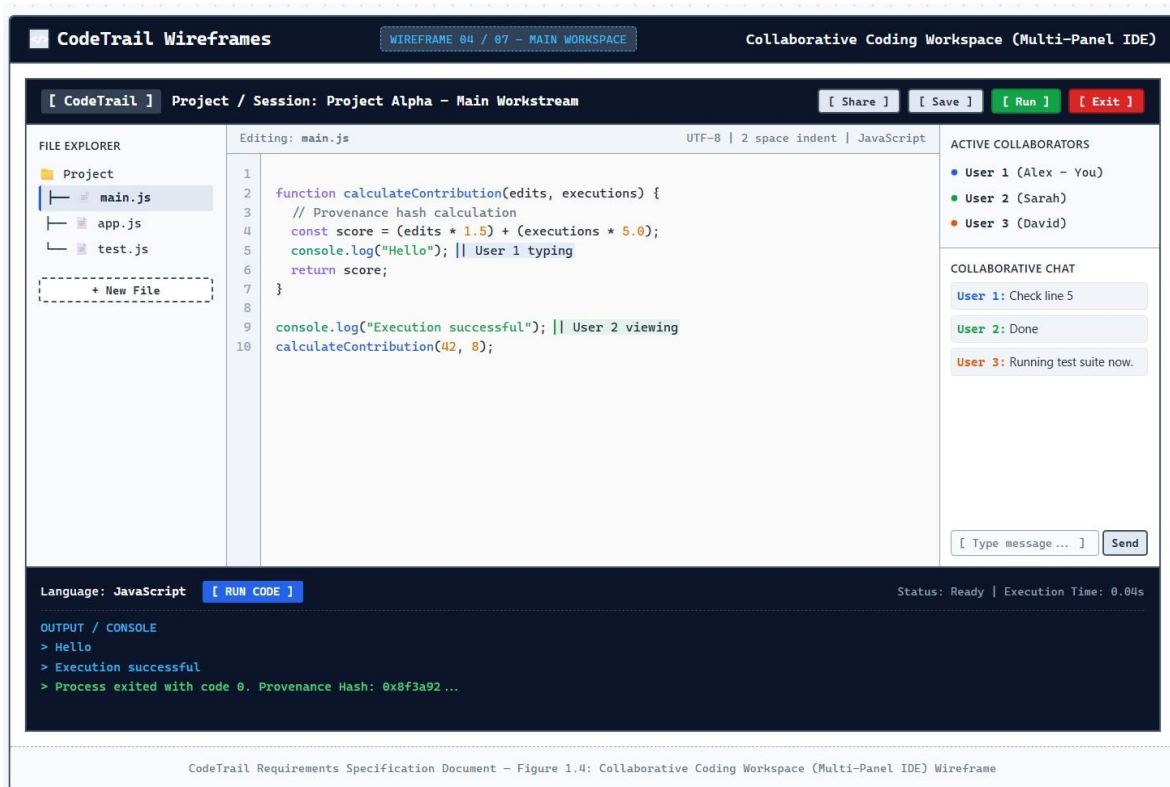


13.4 Wireframe – User Dashboard



13.5 Wireframe – Collaborative Coding Workspace

This is the main wireframe for CodeTrail, because collaborative coding is the core functionality of the application.



CodeTrail Requirements Specification Document – Figure 1.4: Collaborative Coding Workspace (Multi-Panel IDE) Wireframe

Main UI Components

Top Navigation

- CodeTrail logo/name.
- Session name.
- Share/invite option.
- Save option.
- Run option.
- Exit option.
- User/profile controls.

Left Panel

- Project/file explorer.
- File list.
- Create file option.

Center Panel

- Monaco code editor.
- Syntax highlighting. ● Code editing.
- Real-time changes.

Right Panel

- Active users.
- Collaborator information. ● Live collaboration status.

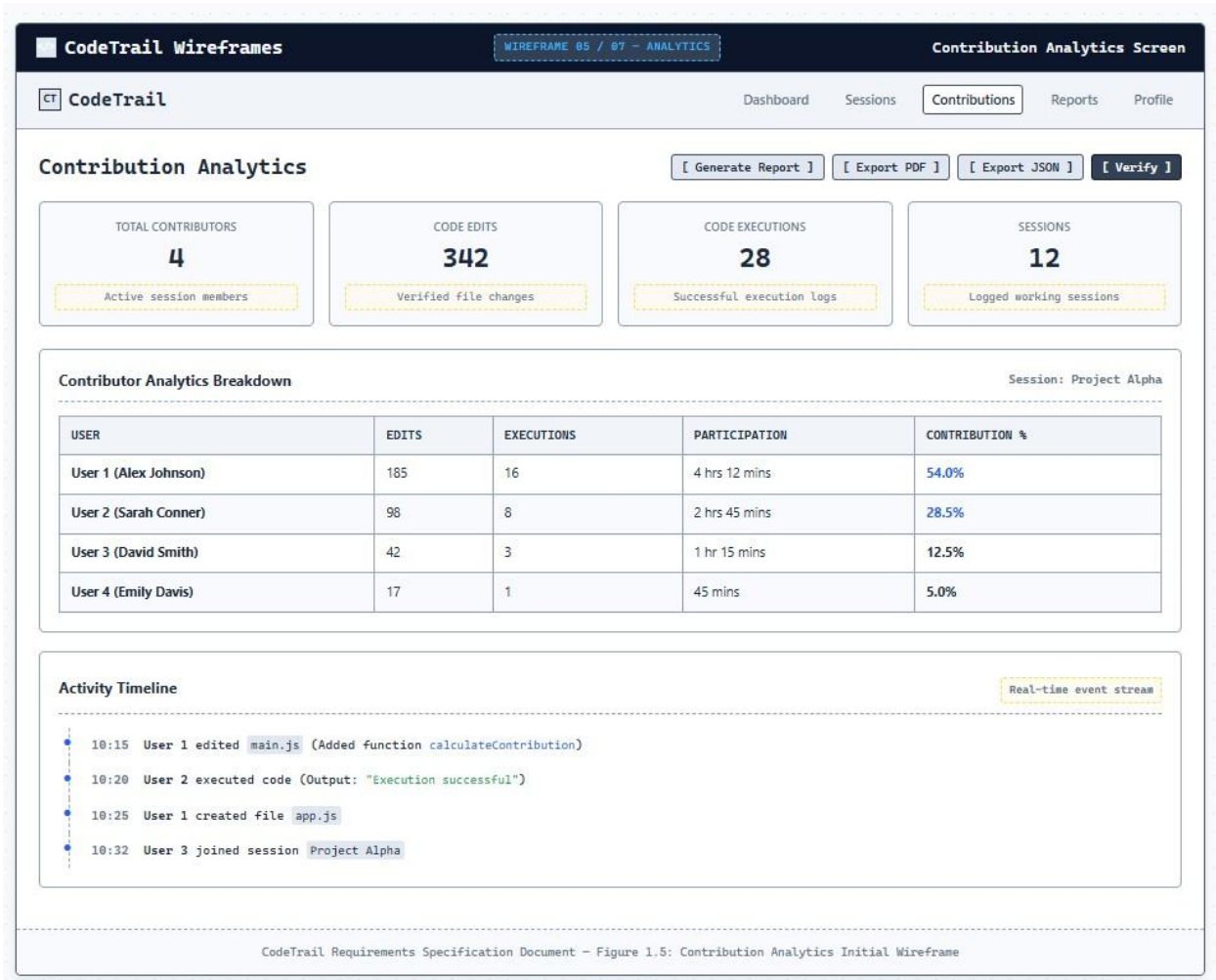
Bottom Panel

- Language selection.
- Run button.
- Execution output. ● Errors.

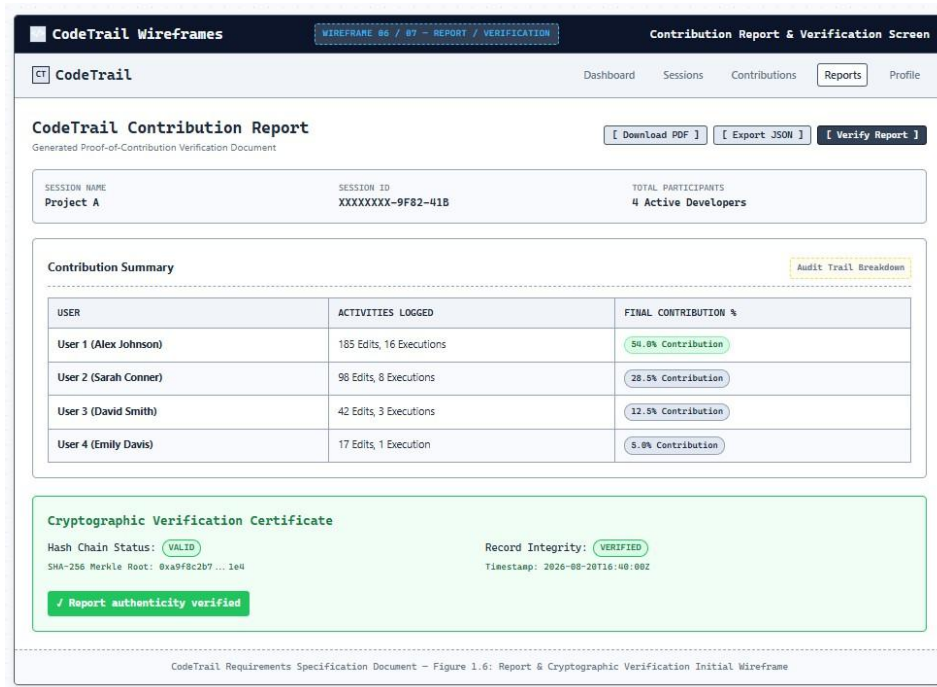
Chat Panel

- Session messages.
- Message input.
- Real-time communication.

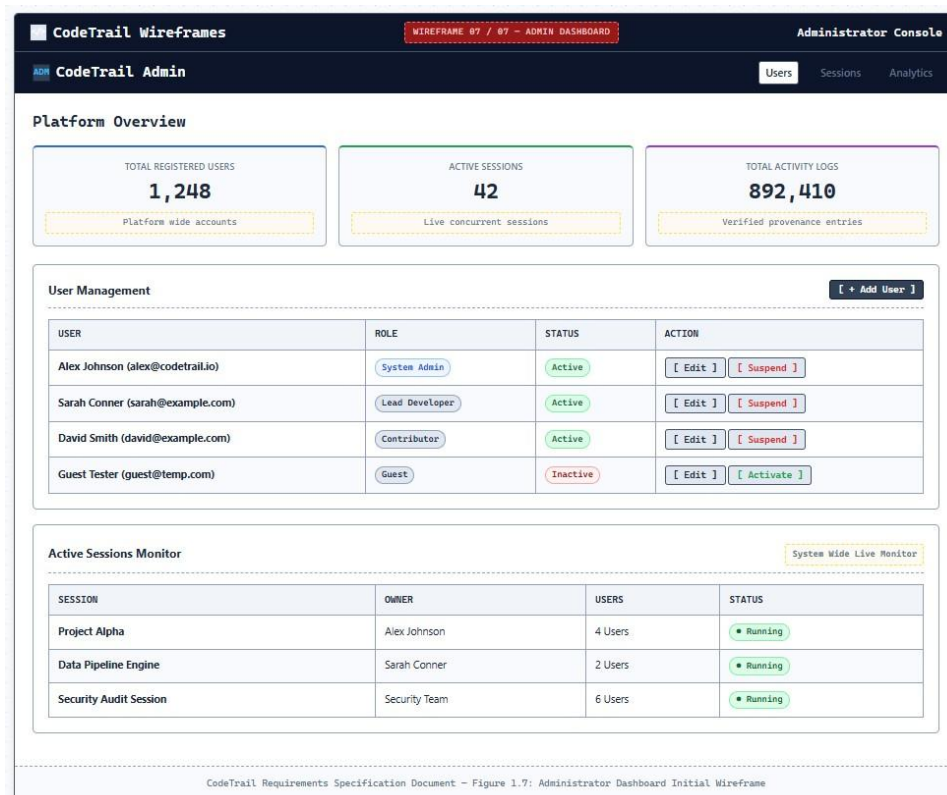
13.6 Wireframe – Contribution Analytics



13.7 Wireframe – Report Verification



13.8 Wireframe – Administrator Dashboard



14. UI/UX Design Requirements

14.1 Simplicity

The interface should allow users to focus on coding and collaboration without unnecessary elements.

14.2 Consistency

Buttons, navigation elements, forms, panels, and information displays should follow a consistent design.

14.3 Visibility

Important information such as:

- Active collaborators.
- Code execution results.
- Errors.
- Session status.
- Contribution information. should remain easily accessible.

14.4 Feedback

The system should provide feedback for:

- Login success/failure.
- Registration success/failure.
- Session creation.
- Session joining.
- Code execution.
- Execution errors.
- Report generation.
- Report verification.

14.5 Responsive Design

The interface should adapt to different screen sizes. The primary coding workspace should remain optimized for desktop/laptop screens because the system requires a multi-pane coding interface.

15. Data Requirements

15.1 User Data The

system shall store:

- User ID.

- User information.
- Authentication information.
- Role.
- Profile information.

15.2 Session Data The

system shall store:

- Session ID.
- Session name.
- Session owner. ● Collaborators.
- Session status.
- Session timestamps.

15.3 Workspace Data

The system shall store:

- Workspace information.
- File information.
- Source code.
- File changes.
- Workspace history.

15.4 Activity Log Data The

system shall store:

- Activity ID.
- User ID.
- Session ID.
- Activity type.
- Activity information.

- Timestamp.
- Current hash.
- Previous hash.

15.5 Contribution Data The

system shall maintain:

- User contribution.
- Code edits.
- Code executions.
- Participation.
- Activity history.

15.6 Report Data

The system shall maintain information required for:

- Contribution reports.
- Activity summaries.
- Timestamps.
- Session details.
- Verification.

16. Security Requirements

The system shall:

1. Authenticate users before accessing protected resources.
2. Hash passwords before database storage.
3. Use JWT for authenticated requests.
4. Restrict access to authorized sessions.
5. Validate user input.
6. Use appropriate security headers.
7. Control cross-origin communication.
8. Securely store application data.

9. Generate SHA-256 hashes for Proof-of-Work logs.
10. Maintain hash-chain integrity.
11. Record administrative activities.
12. Prevent unauthorized access to contribution records.

17. Acceptance Criteria

17.1 Authentication

ID	Acceptance Criteria
AC-01	User can register with valid information.
AC-02	Invalid registration information is rejected.
AC-03	User can log in with valid credentials.
AC-04	Invalid credentials are rejected.

AC-05	Protected resources cannot be accessed without authentication.
-------	--

17.2 Collaborative Workspace

ID	Acceptance Criteria
AC-06	User can create a coding session.
AC-07	Authorized users can join a session.
AC-08	Multiple users can edit the same codebase.
AC-09	Code changes are synchronized in real time.
AC-10	Active collaborators are visible.
AC-11	Cursor information is synchronized.
AC-12	Users can manage workspace files.

17.3 Code Execution

ID	Acceptance Criteria
AC-13	User can select a supported language.
AC-14	User can run code.
AC-15	Program output is displayed.
AC-16	Compilation errors are displayed.
AC-17	Runtime errors are displayed.

AC-18	Execution activity is logged.
-------	-------------------------------

17.4 Proof-of-Work Logging

ID	Acceptance Criteria
AC-19	Significant coding activities are automatically recorded.
AC-20	Activities contain timestamps.
AC-21	Activities are associated with the correct user.
AC-22	SHA-256 hashes are generated.
AC-23	Activity records are connected through hash chaining.
AC-24	Modification of historical records can be detected.

17.5 Contribution Tracking

ID	Acceptance Criteria
AC-25	Individual contributions are recorded.
AC-26	Users can view contribution history.
AC-27	Contribution statistics are generated.
AC-28	User-wise contribution information is displayed.

17.6 Reports

ID	Acceptance Criteria
AC-29	Authorized users can generate reports.
AC-30	Reports contain participant information.
AC-31	Reports contain activity summaries.
AC-32	Reports contain timestamps.
AC-33	Reports can be exported as PDF.
AC-34	Reports can be exported as JSON.
AC-35	Reports can be verified.

17.7 Administration

ID	Acceptance Criteria
AC-36	Administrator can log into the admin dashboard.
AC-37	Administrator can view users.
AC-38	Administrator can manage users.
AC-39	Administrator can monitor active sessions.
AC-40	Administrator can view platform statistics.

AC-41	Administrative actions are recorded.
-------	--------------------------------------

18. Technology Requirements

Layer	Technology
Frontend	React.js
Build Tool	Vite
Styling	Tailwind CSS
Code Editor	Monaco Editor
Backend	Node.js, Express.js

Database	MongoDB, Mongoose
Real-Time Communication	Socket.IO
Code Execution	Piston API
Authentication	JWT
Password Security	bcrypt
Security	Helmet, CORS, Input Validation
Data Integrity	SHA-256
Report Generation	jsPDF
Development	Visual Studio Code
Version Control	Git, GitHub
API Testing	Postman
Database Hosting	MongoDB Atlas

The above technology stack is defined in the CodeTrail project specification.

19. System Constraints

The following constraints apply to the system:

1. CodeTrail is implemented as a web application.
2. Users require an internet connection.
3. Real-time collaboration depends on Socket.IO communication.
4. Code execution depends on the availability of the external execution service.

5. MongoDB is used for persistent data storage.
6. Users access the system through a web browser.
7. The system focuses on collaborative programming rather than replacing complete desktop IDEs.
8. The system does not aim to replace enterprise project-management platforms.
9. Access to private coding sessions requires authentication and authorization.

20. Assumptions

The following assumptions are made:

1. Users have access to a modern web browser.
2. Users have a stable internet connection.
3. Users have valid authentication credentials.
4. Users are authorized to access the sessions they join.
5. The database service is available.
6. The code execution service is available when required.
7. Users have basic programming knowledge.

21. Dependencies

CodeTrail depends on the following technologies and services:

- React.js.
- Vite.
- Tailwind CSS.
- Monaco Editor.
- Node.js.
- Express.js. ● MongoDB.
- Mongoose. ● Socket.IO.
- Piston API.
- JWT.
- bcrypt.
- Helmet.
- CORS.

- jsPDF.
- MongoDB Atlas.
- GitHub.

22. Future Enhancements

Potential future enhancements include:

- Additional programming language support.
- Advanced code review.
- More detailed contribution analytics.
- Enhanced session replay.
- Advanced project management.
- Additional authentication providers.
- Advanced notifications.
- Improved scalability for large collaboration sessions.
- Integration with version-control platforms. ● AI-assisted coding features.

These features are considered future enhancements and are not part of the core requirements defined for the current version.

23. Conclusion

CodeTrail is designed to combine real-time collaborative programming with transparent and verifiable contribution tracking.

The system enables multiple users to collaborate on source code, execute programs, communicate within coding sessions, and automatically maintain records of their individual activities.

The Requirements Specification Document defines the complete requirements necessary for the development and evaluation of CodeTrail, including:

- Functional requirements.
- Non-functional requirements.
- User stories.
- Use cases.
- Use case diagram.
- Interface requirements.
- External interface requirements.
- Initial UI/UX wireframes.
- Data requirements.
- Security requirements.
- Acceptance criteria.
- Technology requirements.
- System constraints.
- Assumptions and dependencies.

The core Proof-of-Work mechanism provides a tamper-evident contribution history by linking activity records using SHA-256 hashing. This allows contribution records to be used as evidence of individual participation while maintaining transparency and accountability.

CodeTrail therefore provides a unified platform for collaborative coding, real-time communication, browser-based code execution, contribution tracking, analytics, report generation, and contribution verification.